

Strategie di ricerca informata

Claudio Gallicchio, 2025/26
claudio.gallicchio@unipi.it



Intelligenza artificiale

Un approccio moderno

Volume 1

Quarta edizione

Stuart Russell, Peter Norvig
a cura di Francesco Amigoni



MyLab Codice per accedere
alla piattaforma

Riferimenti

AIMA Cap. 3

Sez. 3.5, 3.6

Strategie di ricerca informata o
euristica

Ricerca informata

Idea fondamentale: usare conoscenza specifica del dominio applicativo per **fornire suggerimenti su dove si potrebbe trovare l'obiettivo**

Ricerca informata

Idea fondamentale: usare conoscenza specifica del dominio applicativo per **fornire suggerimenti su dove si potrebbe trovare l'obiettivo**

Funzione euristica $h(n)$

$h(n)$ = costo *stimato* del cammino
meno costoso dallo stato del
nodo n a uno stato obiettivo

Ricerca best-first greedy (golosa)

Idea: espandere **prima il nodo con il valore più basso di $h(n)$** , cioè quello che appare più vicino all'obiettivo.

Funzione di valutazione: $f(n) = h(n)$

Es. distanza in linea d'aria h_{DLA}

$h_{DLA}(n)$ = distanza in linea d'aria tra la città corrispondente ad n e Bucarest.

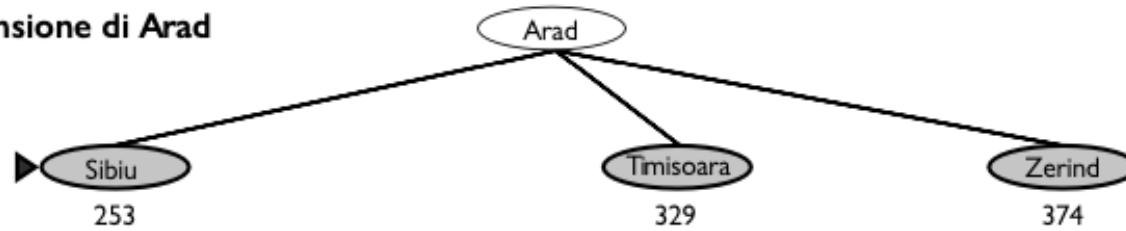
Arad	366	Mehadia	241
Bucarest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Nota: i valori dell'euristica non sono ottenibili direttamente dalla descrizione del problema (funzioni Azioni($\cdot$) e Risultato($\cdot$))

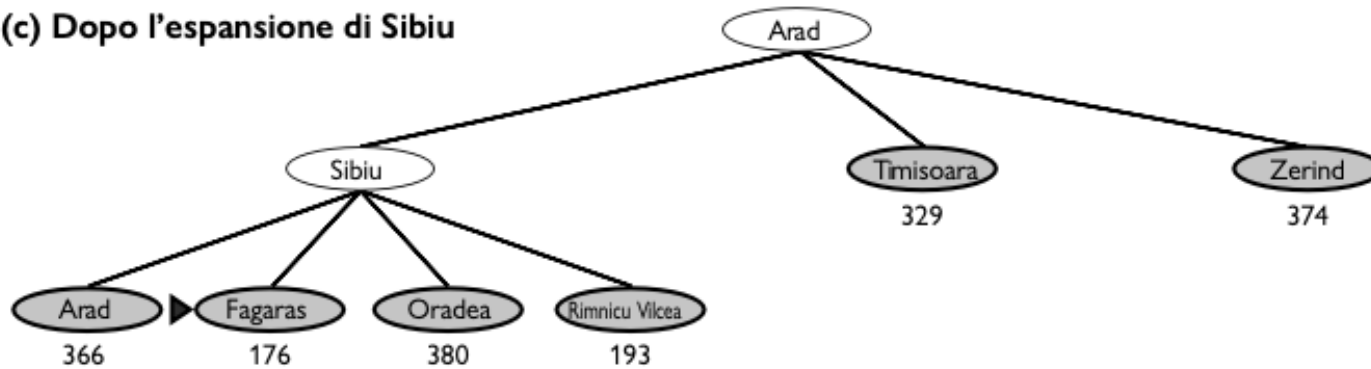
(a) Stato iniziale



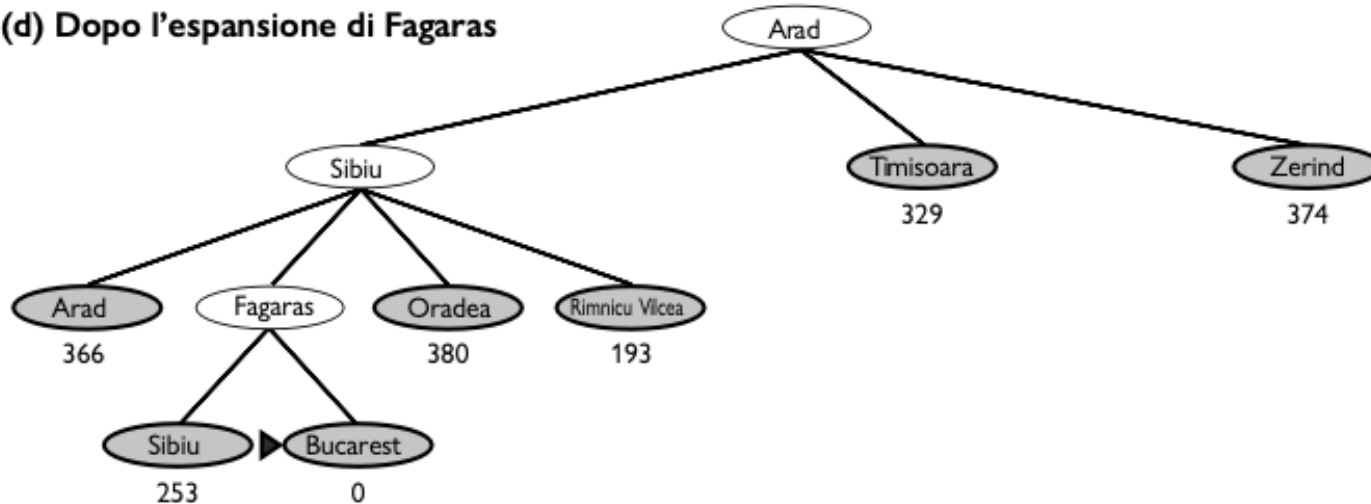
(b) Dopo l'espansione di Arad



(c) Dopo l'espansione di Sibiu

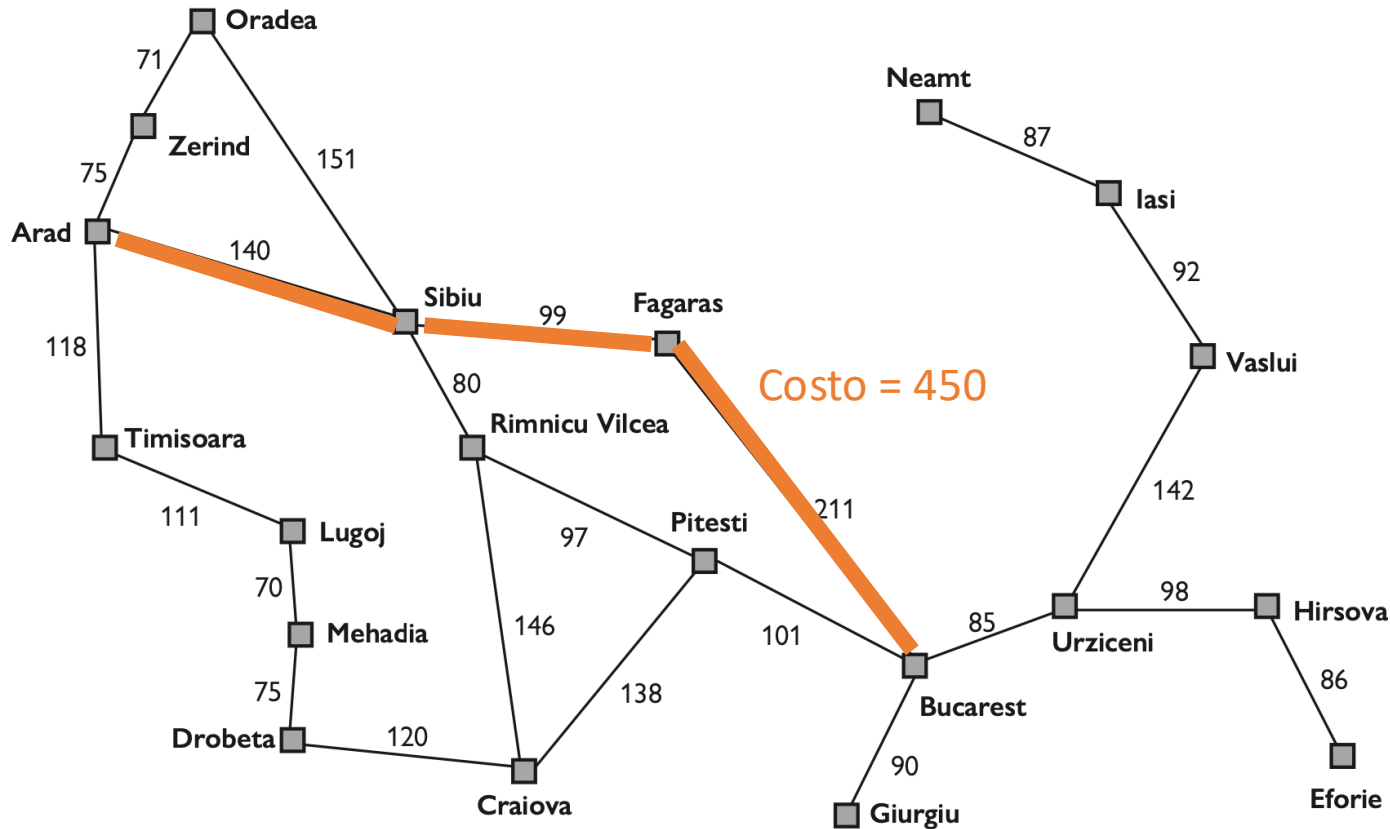


(d) Dopo l'espansione di Fagaras



Es. distanza in
linea d'aria h_{DLA}

Es. distanza in linea d'aria h_{DLA}

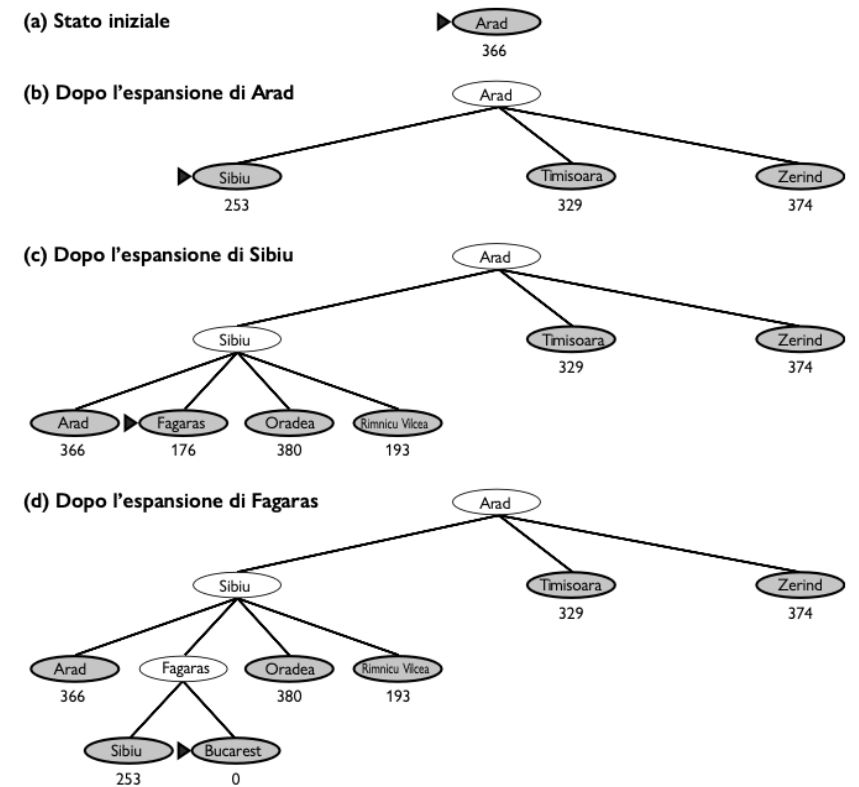


(a) Stato iniziale

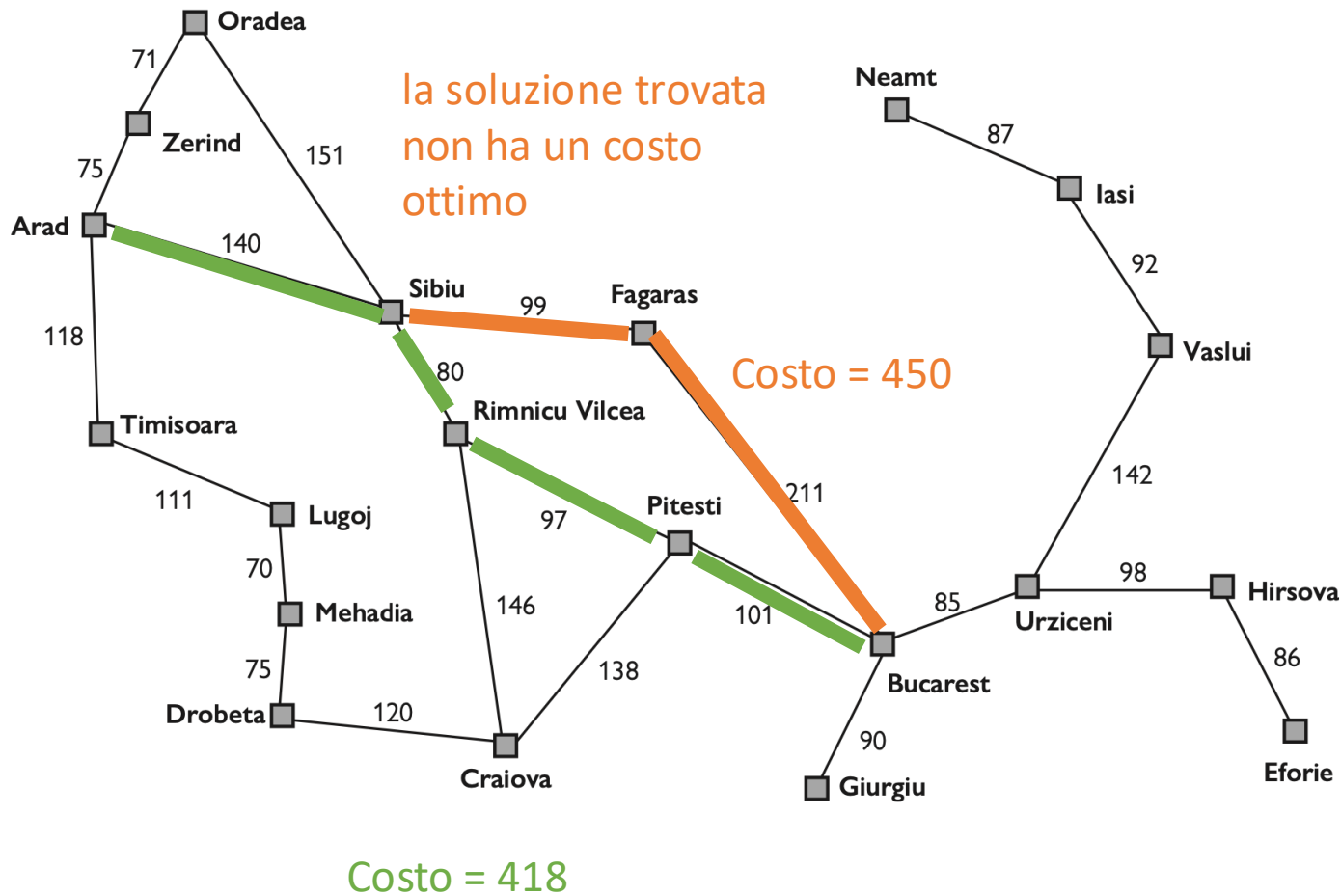
(b) Dopo l'espansione di Arad

(c) Dopo l'espansione di Sibiu

(d) Dopo l'espansione di Fagaras



Es. distanza in linea d'aria h_{DLA}

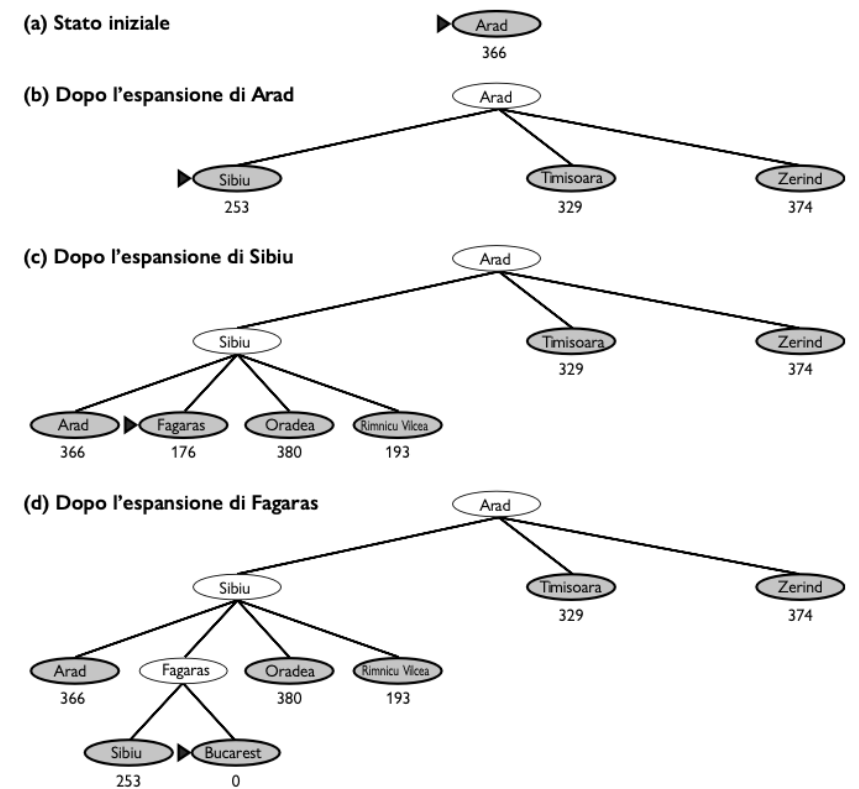


(a) Stato iniziale

(b) Dopo l'espansione di Arad

(c) Dopo l'espansione di Sibiu

(d) Dopo l'espansione di Fagaras



Ricerca best-first greedy: analisi

- La ricerca best-first greedy su grafo è completa in spazi di stati finiti, ma non in quelli infiniti.
- Non ottima rispetto al costo
- Complessità spaziale e temporale $O(|V|)$
 - con buona funzione euristica si può ridurre fino a $O(b^m)$

Ricerca A^*

A «star», o A «stella»

Ricerca A*

Ricerca best-first che usa la seguente funzione di valutazione:

$$f(n) = g(n) + h(n)$$

Ricerca A*

Ricerca best-first che usa la seguente funzione di valutazione:

costo del cammino dal nodo
iniziale al nodo n

$$f(n) = g(n) + h(n)$$

costo stimato del cammino migliore
che continua da n fino a un nodo
obiettivo

costo stimato del cammino più
breve da n a uno stato obiettivo

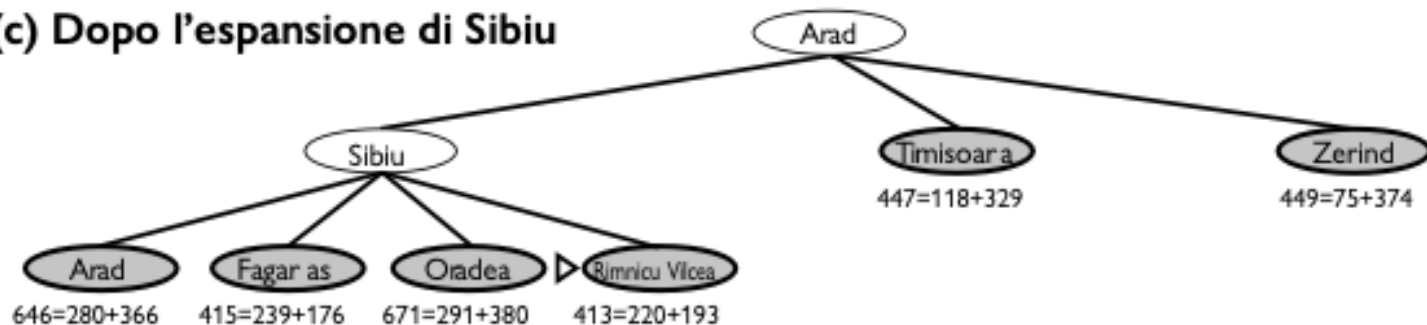
(a) Stato iniziale



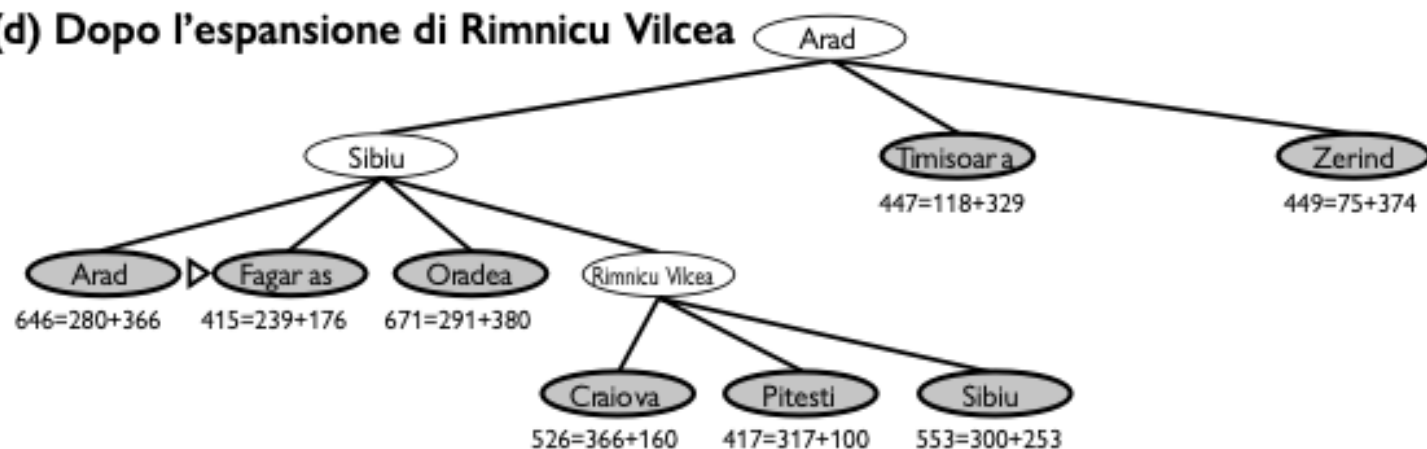
(b) Dopo l'espansione di Arad



(c) Dopo l'espansione di Sibiu



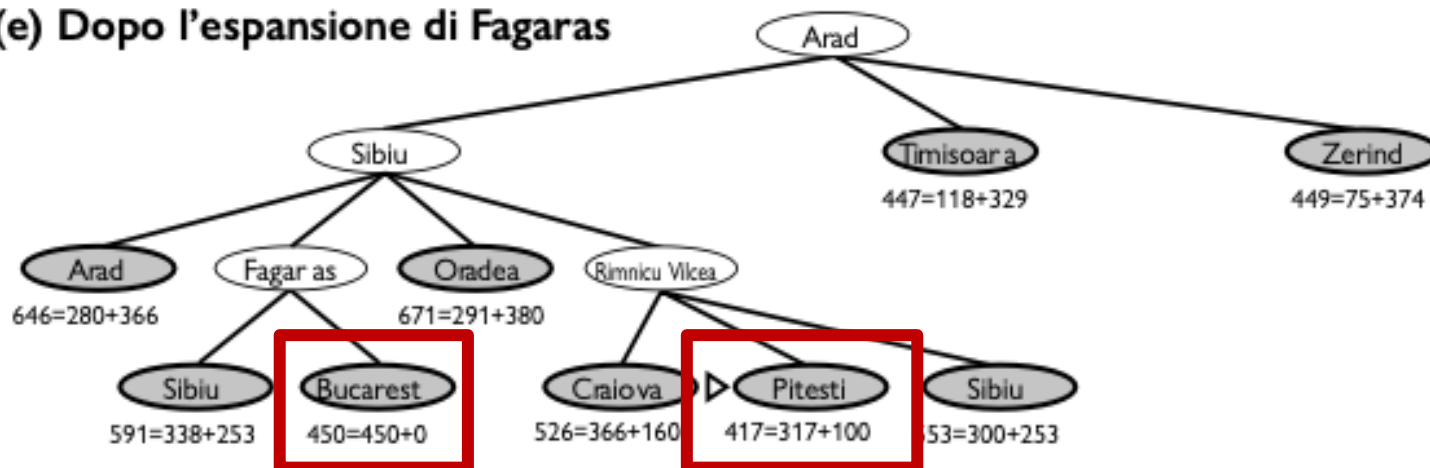
(d) Dopo l'espansione di Rimnicu Vilcea



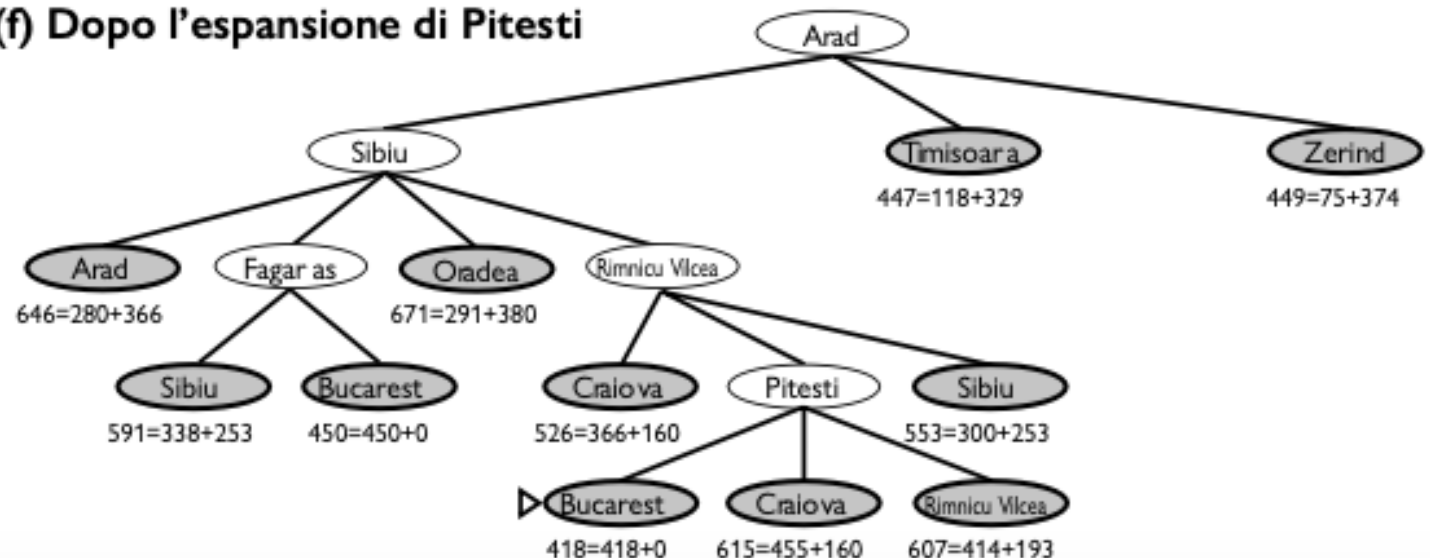
Esempio

Partire da Arad e raggiungere Bucarest

(e) Dopo l'espansione di Fagaras



(f) Dopo l'espansione di Pitesti



Qui Bucarest appare sulla frontiera ma non viene selezionata per l'espansione perché ha $f = 450$, mentre per Pitesti $f = 417$

Esempio

Partire da Arad e raggiungere Bucarest

Ricerca A*: analisi

- Completa
 - tutti i costi di azione sono $> \varepsilon > 0$
 - lo spazio degli stati ha una soluzione o è finito
- Ottima rispetto al costo se l'euristica $h(n)$ è ammissibile

Euristica ammissibile

Un'euristica è ammissibile se **non sovrastima mai il costo** per raggiungere un obiettivo.

- «ottimista»

Se $h^*(n)$ è il costo minimo di un cammino da n fino a un goal, allora $h(n) \leq h^*(n)$

Se $h(\cdot)$ è ammissibile allora A^* è ottimo rispetto al costo

Dim per assurdo.

A^* restituisce un cammino di costo C , ma esiste una soluzione di costo $C^* < C$.

Allora certamente esiste un nodo n sul cammino ottimo che non è stato espanso.

Se avessimo espanso tutti i nodi sul cammino Ottimo avremmo restituito la soluzione ottima

Se $h(\cdot)$ è ammissibile allora A^* è ottimo rispetto al costo Dim per assurdo.

Denotiamo con:

- $g^*(n)$ costo del cammino ottimo *dall'inizio fino ad n*
- $h^*(n)$ costo del cammino ottimo *da n fino ad un obiettivo*
- $f(n) > C^*$ Altrimenti n sarebbe stato espanso
- $f(n) = g(n) + h(n)$ Per definizione
- $f(n) = g^*(n) + h(n)$ Siccome n si trova su un cammino ottimo
- $f(n) \leq g^*(n) + h^*(n)$ Siccome $h(n)$ è ammissibile allora $h(n) \leq h^*(n)$
- $f(n) \leq C^*$ Per definizione, $C^* = g^*(n) + h^*(n)$

Quindi A^* restituisce solo cammini ottimali rispetto al costo.

Euristica consistente

Un'euristica $h(n)$ è consistente se, per ogni nodo n , e per ogni successore n' di n generato da un'azione a , risulta:

$$h(n) \leq c(n, a, n') + h(n')$$

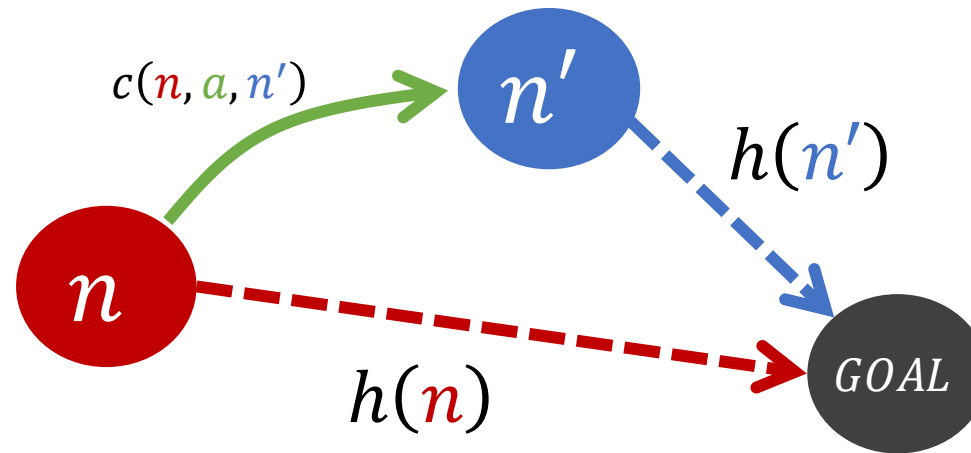
Costo dell'azione per
andare da n a n'

Stima euristica in $h(n')$

Euristica consistente

Un'euristica $h(n)$ è consistente se, per ogni nodo n , e per ogni successore n' di n generato da un'azione a , risulta:

$$h(n) \leq c(n, a, n') + h(n')$$

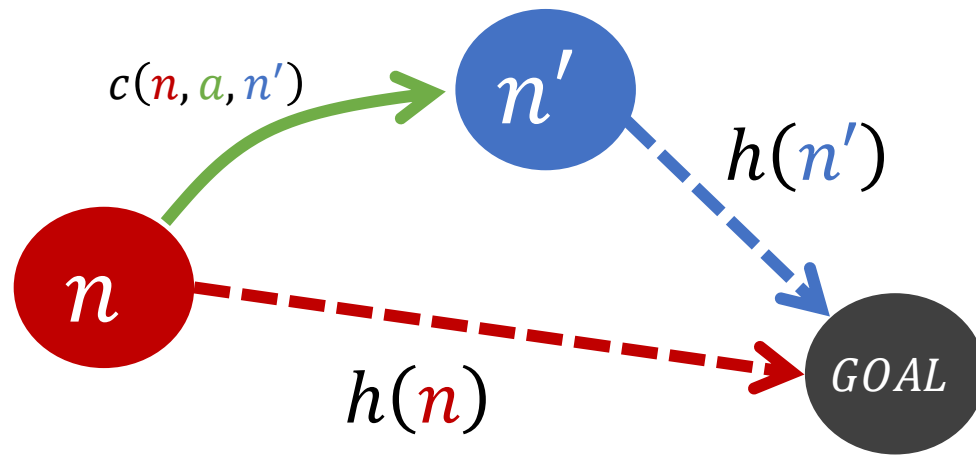


Disuguaglianza triangolare

Euristica consistente

Un'euristica $h(n)$ è consistente se, per ogni nodo n , e per ogni successore n' di n generato da un'azione a , risulta:

$$h(n) \leq c(n, a, n') + h(n')$$



Nota: se h è un'euristica consistente, la funzione di valutazione $f = g + h$ **non decresce mai lungo i cammini**. In questo caso si dice anche che h è un'euristica **monotona**.

Disuguaglianza triangolare

Euristica consistente

Se un'euristica è consistente allora è anche ammissibile
(non vale il vice versa)

Quindi, A^* con un'euristica consistente è ottima rispetto al costo.

- es. la distanza in linea d'aria è consistente

A* con euristica consistente

- Ottimo rispetto al costo
- La prima volta che raggiungiamo uno stato sarà su un cammino ottimo
 - non dovremo mai aggiungere di nuovo uno stato alla frontiera
 - non dovremo mai modificare un elemento in raggiunti

A* con euristica inconsistente

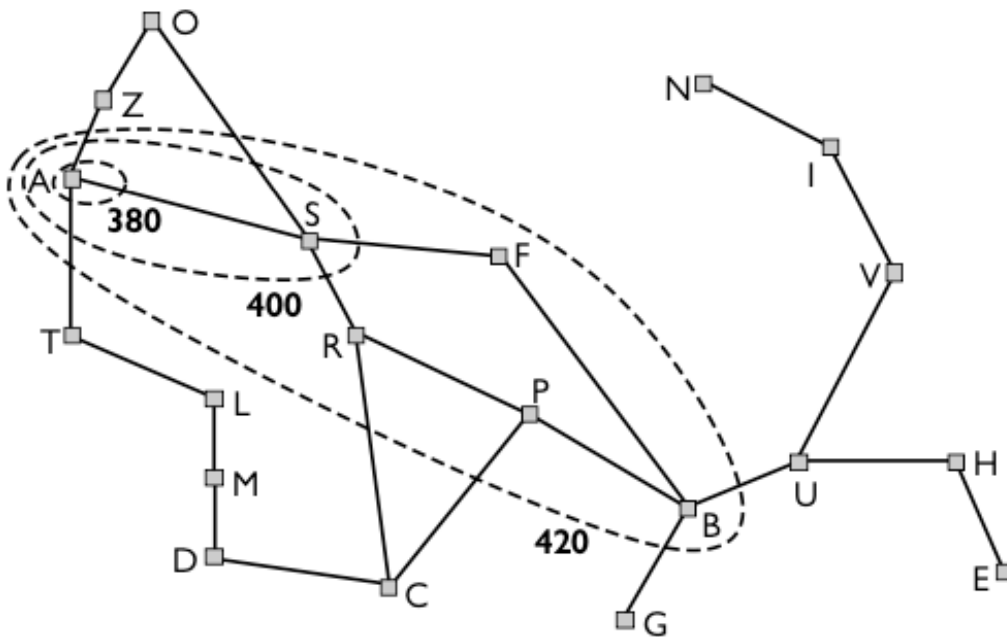
- Possiamo trovarci con più cammini che raggiungono lo stesso stato
- Se ogni cammino nuovo ha costo inferiore al precedente finiremo con l'avere più nodi corrispondenti a quello stato sulla frontiera
 - aggravio di costo spaziale e temporale

A* con euristica inammissibile

Può essere ottima rispetto al costo in due casi

- Esiste un cammino ottimo rispetto al costo lungo cui $h(n)$ è ammissibile per tutti i nodi sul cammino
 - il cammino verrà trovato anche se l'euristica non è ammissibile fuori da esso
- Se la soluzione ottima ha costo C^* e la seconda soluzione migliore ha costo C_2 , e se $h(n)$ sovrastima i costi, ma al massimo di una quantità $\leq C_2 - C^*$

Confini della ricerca



A* espande il nodo della frontiera di costo f minimo.

A* si espande a partire dal nodo iniziale aggiungendo nodi in **bande concentriche di f -costo crescente**.

Nota: confini nella ricerca a costo uniforme sono concentrici, ma circolari.

Nella ricerca A* con una buona euristica, **i confini si allungano verso uno stato obiettivo**, focalizzandosi intorno a un cammino ottimo.

Caratteristiche di A^*

Se C^* è il costo del cammino della soluzione ottima:

- A^* **espande tutti i nodi** che possono essere raggiunti dallo stato iniziale **su un cammino in cui** per ogni nodo si ha $f(n) < C^*$
 - questi nodi sono i «nodi certamente espansi»
- A^* **può espandere** alcuni dei nodi sul confine obiettivo (dove $f(n) = C^*$) prima di selezionare il nodo obiettivo.
- A^* **non espande** alcun nodo con $f(n) > C^*$

A* ottimamente efficiente

A* con un'euristica consistente è ottimamente efficiente.

Qualsiasi algoritmo che estende i cammini di ricerca a partire dallo stato iniziale e usa la stessa euristica deve espandere tutti i nodi che sono certamente espansi da A*.

Nota: sul confine obiettivo si può essere più o meno «fortunati».

Potatura di sottoalberi: aspetto focale per l'efficienza di A*. Non dobbiamo generare e tenere in memoria sotto-alberi inutili a trovare una soluzione ottima.

Ricerca soddisfacente

A* espande un grande numero di nodi.

A volte va bene accettare **soluzioni subottime**, ma «**sufficientemente buone**» (**soddisfacenti** / satisficing), purchè si risparmi esplorando un numero minore di nodi.

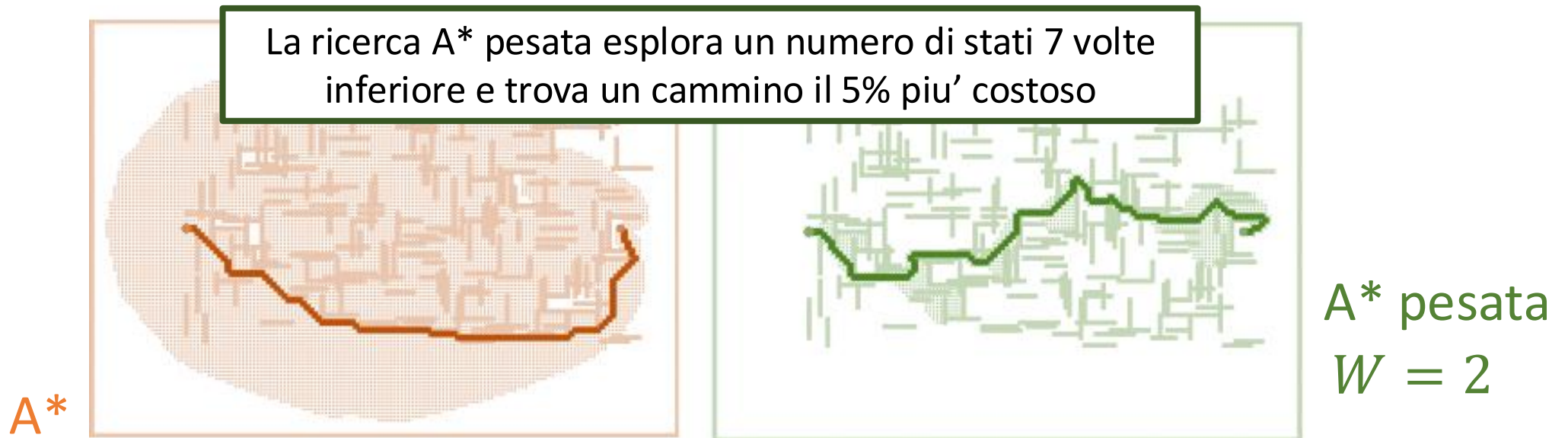
Possiamo consentire ad A* di usare un'euristica inammissibile (che potrebbe sovrastimare).

- Rischiamo di trovare una soluzione subottima
- Euristica potrebbe essere più accurata: possiamo risparmiare sul numero di nodi espansi

Ricerca A* pesata

Diamo un peso maggiore al valore dell'euristica.

$$f(n) = g(n) + W h(n), \text{ per } W > 1.$$



Ricerca A* pesata

Diamo un peso maggiore al valore dell'euristica.

$$f(n) = g(n) + W h(n), \text{ per } W > 1.$$

Se la soluzione ottima ha costo C^* , troverà una soluzione di costo compreso tra C^* e $W \times C^*$.

Ricerca A* pesata

Si può considerare come una generalizzazione delle altre.

Ricerca A*:	$g(n) + h(n)$	$(W = 1)$
Ricerca a costo uniforme:	$g(n)$	$(W = 0)$
Ricerca best-first greedy:	$h(n)$	$(W = \infty)$
Ricerca A* pesata:	$g(n) + W \times h(n)$	$(1 < W < \infty)$

Come la ricerca best-first greedy, focalizza la ricerca verso un obiettivo. Ma non ignorerà mai completamente il costo del cammino (pruning dei cammini con scarsi progressi ad alti costi)

Algoritmi di ricerca subottima

- **Ricerca a subottimalità limitata**: cerchiamo una soluzione con garanzia che si trovi entro un fattore costante W del costo ottimo
 - es. A* pesata.
- **Ricerca a costo limitato**: cerchiamo una soluzione il cui costo sia inferiore a una costante C .
- **Ricerca a costo illimitato**: accettiamo una soluzione di qualsiasi costo purché riusciamo a trovarla rapidamente.
 - es. speedy search: best-first greedy con $h(n)$ = numero stimato di azioni per raggiungere un obiettivo.

Ricerca con memoria limitata

La problematica principale della ricerca A^* è legato all'uso della memoria.

- La memoria è divisa tra frontiera e raggiunti
- Uno stato che si trova sulla frontiera è memorizzato in due posizioni
 - come nodo della frontiera (per poterlo espandere)
 - come elemento nella tabella dei raggiunti (sappiamo se l'abbiamo già visitato)

Ricerca con memoria limitata

Trucco #1: gli stati sono presenti solo in una delle due posizioni (frontiera o raggiunti)

Ricerca con memoria limitata

Trucco #1: gli stati sono presenti solo in una delle due posizioni (frontiera o raggiunti)

Trucco #2: rimuovere gli stati da raggiunti quando si può dimostrare che non servono più.

- Garantire che le azioni muovano verso l'esterno della frontiera (o verso un altro stato di frontiera)
 - Es. tramite proprietà di separazione, divieto di azioni reversibili
 - Si può eliminare la tabella raggiunti controllando che nella frontiera non ci siano cammini ridondanti
- Contare quante volte uno stato è stato raggiunto e rimuoverlo dai raggiunti se non ci sono più modi per raggiungerlo
 - Es griglia dove una posizione può essere raggiunta solo dalle caselle adiacenti

Ricerca beam

Limitare la dimensione della **frontiera**, mantenendo solo i k nodi con i migliori costi f

- Incompleta e subottima
- Esplora solo un settore dei confini concentrici
 - quelli che contengono i migliori k candidati
- Alternativa: mantenere ogni nodo il cui **costo f** rientra in un fattore δ del miglior costo f .

Ricerca A* ad approfondimento iterativo IDA*

Ricerca ad **approfondimento iterativo combinata con A***.

- Combina i vantaggi di A* senza dover tenere in memoria gli stati raggiunti.
- La **soglia è sul valore della funzione** $f = g + h$ invece che sulla profondità.
- Per ogni iterazione:
 - ricerca esaustiva (in profondità) fino ad un costo f_l
 - quando viene trovato un nodo n con costo $f(n) > f_l$, termina iterazione e ricomincia la successiva con $f_l = f(n)$.

Ricerca Best-first ricorsiva (RBFS)

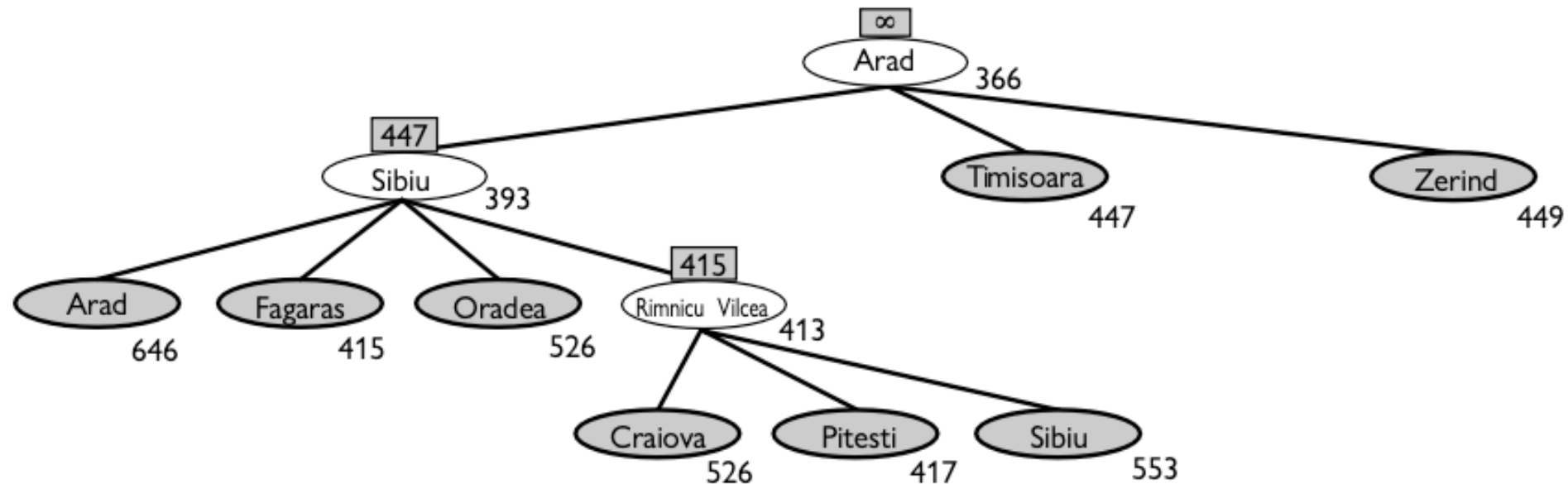
Ricerca in BF con **interruzione** quando f supera un valore f_{limite} .

- f_{limite} = il costo trovato sul **miglior cammino alternativo**
- se il nodo corrente supera questo limite, la ricorsione torna al cammino alternativo
 - Durante il ritorno, sostituisce il valore f di ogni nodo lungo il cammino con il miglior f dei suoi nodi figli
 - ✓ valore di backup che indica il valore della foglia migliore in un cammino abbandonato (per eventualmente tornarci)

Ricerca Best-first ricorsiva (RBFS)

Durante il ritorno, sostituisce il valore f di ogni nodo lungo il cammino con il miglior f dei suoi nodi figli

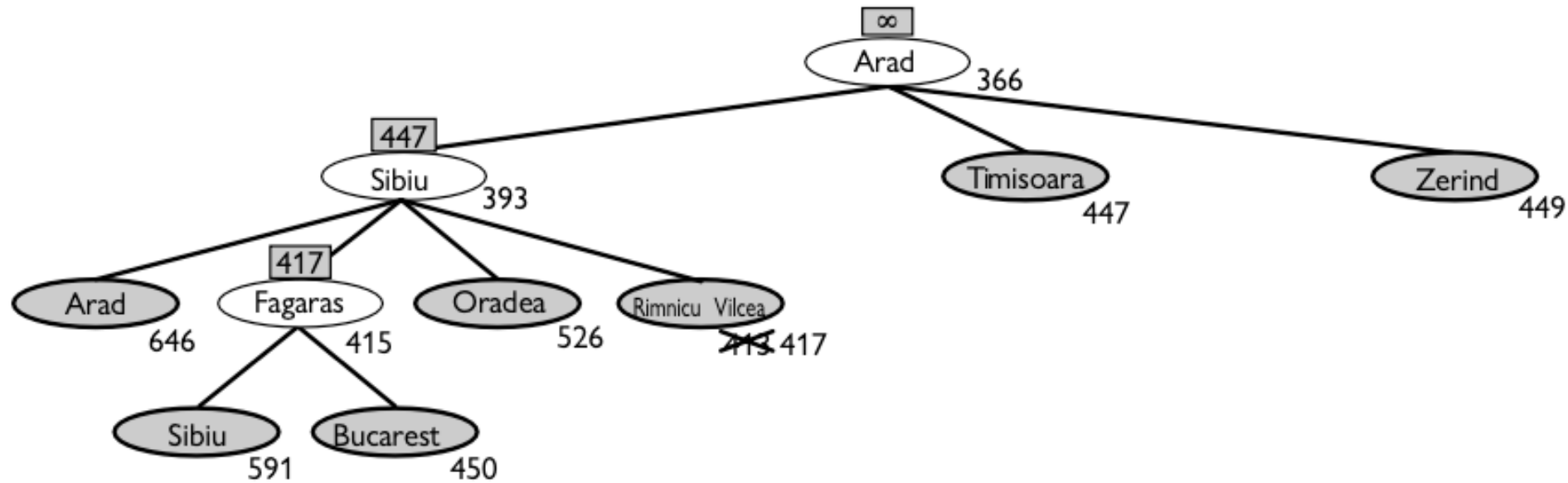
- ✓ valore di backup che indica il valore della foglia migliore in un cammino abbandonato (per eventualmente tornarci)



Ricerca Best-first ricorsiva (RBFS)

Durante il ritorno, sostituisce il valore f di ogni nodo lungo il cammino con il miglior f dei suoi nodi figli

- ✓ valore di backup che indica il valore della foglia migliore in un cammino abbandonato (per eventualmente tornarci)



Memory bounded A* (SMA*)

- Idea: usare al meglio (tutta) la memoria disponibile
- SMA* procede come A* fino ad esaurimento della memoria disponibile
- Poi, «dimentica» il nodo peggiore, memorizzando nel nodo del padre il valore del nodo dimenticato
- A parità di f : espande la foglia migliore più recente e cancella la foglia peggiore più vecchia
- Ottimale se il cammino soluzione sta in memoria

Funzioni euristiche

Esempio: rompicapo a 8 tasselli

7	2	4
5		6
8	3	1

Stato iniziale

	1	2
3	4	5
6	7	8

Stato obiettivo

Esempio euristica

h_1 = numero di tasselli fuori posto
(spazi vuoti non inclusi).

$$h_1 = 8$$

7	2	4
5		6
8	3	1

stato iniziale

	1	2
3	4	5
6	7	8

stato obiettivo

Esempio euristica

7	2	4
5		6
8	3	1

stato iniziale

	1	2
3	4	5
6	7	8

stato obiettivo

h_1 = numero di tasselli fuori posto
(spazi vuoti non inclusi).

Ammissibile: ogni tassello fuori posto richiederà
almeno 1 spostamento.

Esempio euristica

h_2 = somma delle distanze di tutti i tasselli dalla loro posizione corrente alla configurazione obiettivo.

Distanza Manhattan (solo in orizzontale e verticale).

$$h_2 = 18$$

7	2	4
5		6
8	3	1

stato iniziale

	1	2
3	4	5
6	7	8

stato obiettivo

Esempio euristica

7	2	4
5		6
8	3	1

stato iniziale

	1	2
3	4	5
6	7	8

stato obiettivo

h_2 = somma delle distanze di tutti i tasselli dalla loro posizione corrente alla configurazione obiettivo.

Distanza Manhattan (solo in orizzontale e verticale).

Ammissibile: ogni mossa può al massimo avvicinare un tassello di 1 passo più vicino al suo obiettivo.

Fattore di ramificazione effettivo

N = numero totale di nodi generato da A^*

d = profondità dell'albero generato da A^*

b^* = fattore di ramificazione che un albero uniforme di profondità d dovrebbe avere per contenere $N + 1$ nodi.

$$N + 1 = 1 + b^* + (b^*)^2 + \dots + (b^*)^d$$

Fattore di ramificazione effettivo: esempio

Ricerca in ampiezza
non informata

Lunghezza
della
soluzione

d	Costo di ricerca (nodi generati)			Fattore di ramificazione effettivo		
	RIA	$A^*(h_1)$	$A^*(h_2)$	RIA	$A^*(h_1)$	$A^*(h_2)$
6	128	24	19	2,01	1,42	1,34
8	368	48	31	1,91	1,40	1,30
10	1033	116	48	1,85	1,43	1,27
12	2672	279	84	1,80	1,45	1,28
14	6783	678	174	1,77	1,47	1,31
16	17270	1683	364	1,74	1,48	1,32
18	41558	4102	751	1,72	1,49	1,34
20	91493	9905	1318	1,69	1,50	1,34
22	175921	22955	2548	1,66	1,50	1,34
24	290082	53039	5733	1,62	1,50	1,36
26	395355	110372	10080	1,58	1,50	1,35
28	463234	202565	22055	1,53	1,49	1,36

Fattore di ramificazione effettivo: esempio

d	Costo di ricerca (nodi generati)			Fattore di ramificazione effettivo		
	RIA	$A^*(h_1)$	$A^*(h_2)$	RIA	$A^*(h_1)$	$A^*(h_2)$
6	128	24	19	2,01	1,42	1,34
8	368	48	31	1,91	1,40	1,30
10	1033	116	48	1,85	1,43	1,27
12	2672	279	84	1,80	1,45	1,28
14	6783	678	174	1,77	1,47	1,31
16	17270	1683	364	1,74	1,48	1,32
18	41558	4102	751	1,72	1,49	1,34
20	91493	9905	1318	1,69	1,50	1,34
22	175921	22955	2548	1,66	1,50	1,34
24	290082	53039	5733	1,62	1,50	1,36
26	395355	110372	10080	1,58	1,50	1,35
28	463234	202565	22055	1,53	1,49	1,36

h_2 è migliore di h_1

Fattore di ramificazione effettivo: esempio

d	Costo di ricerca (nodi generati)			Fattore di ramificazione effettivo		
	RIA	$A^*(h_1)$	$A^*(h_2)$	RIA	$A^*(h_1)$	$A^*(h_2)$
6	128	24	19	2,01	1,42	1,34
8	368	48	31	1,91	1,40	1,30
10	1033	116	48	1,85	1,43	1,27
12	2672	279	84	1,80	1,45	1,28
14	6783	678	174	1,77	1,47	1,31
16	17270	1683	364	1,74	1,48	1,32
18	41558	4102	751	1,72	1,49	1,34
20	91493	9905	1318	1,69	1,50	1,34
22	175921	22955	2548	1,66	1,50	1,34
24	290082	53039	5733	1,62	1,50	1,36
26	395355	110372	10080	1,58	1,50	1,35
28	463234	202565	22055	1,53	1,49	1,36

entrambe h_2 ed h_1 sono migliori rispetto a non avere alcuna euristica

Fattore di ramificazione effettivo: esempio

Costo di ricerca (nodi generati)				Fattore di ramificazione effettivo		
d	RIA	$A^*(h_1)$	$A^*(h_2)$	RIA	$A^*(h_1)$	$A^*(h_2)$
6	128	24	19	2,01	1,42	1,34
8	368	48	31	1,91	1,40	1,30
10	1033	116	48	1,85	1,43	1,27
12	2672	279	84	1,80	1,45	1,28
14	6783	678	174	1,77	1,47	1,31
16	17270	1683	364	1,74	1,48	1,32
18	41558	4102	751	1,72	1,49	1,34
20	91493	9905	1318	1,69	1,50	1,34
22	175921	22955	2548	1,66	1,50	1,34
24	290082	53039	5733	1,62	1,50	1,36
26	395355	110372	10080	1,58	1,50	1,35
28	463234	202565	22055	1,53	1,49	1,36

una buona euristica dovrebbe avere un valore di b^* vicino ad 1.

Dominazione

h_2 domina h_1 quando per ogni nodo n , risulta

$$h_1(n) \leq h_2(n)$$

Dominazione vuol dire maggiore efficienza.

A* con euristica h_2 non espanderà più nodi di A* con euristica h_1

- recall: A* espande certamente n se $f(n) < C^*$

Dominazione

h_2 domina h_1 quando per ogni nodo n , risulta

$$h_1(n) \leq h_2(n)$$

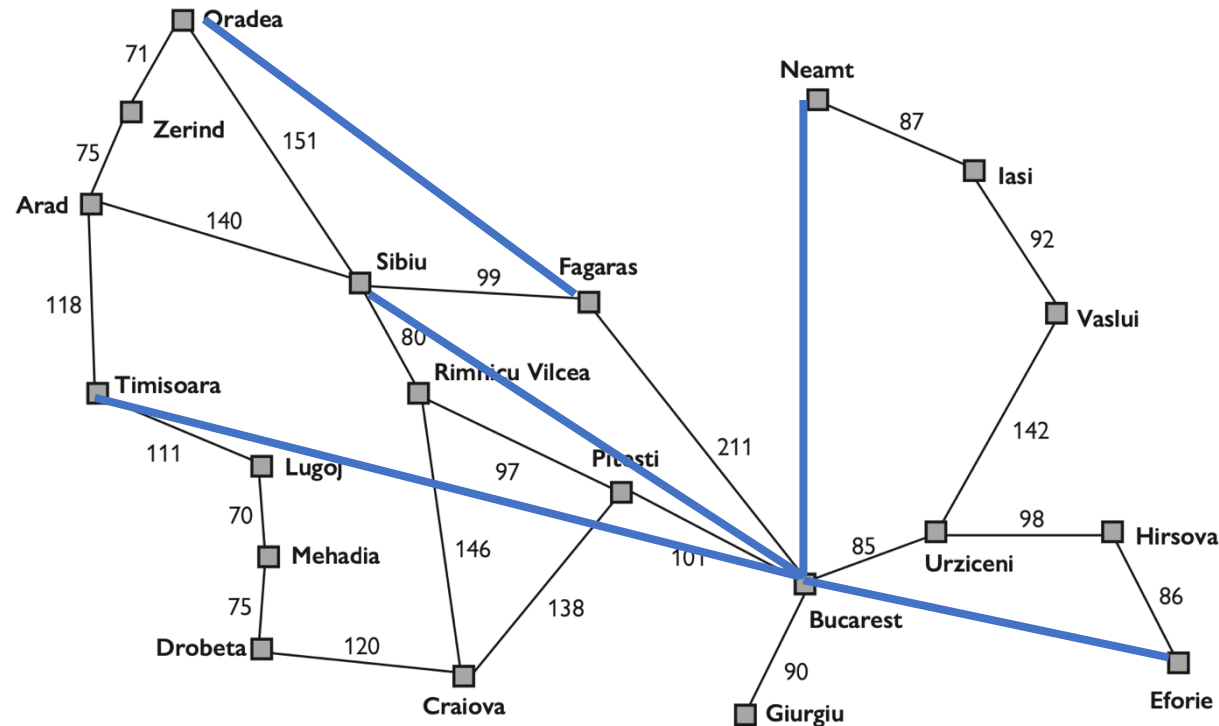
Generalmente meglio usare un'euristica con valori più alti

- a patto che sia consistente (quindi ammissibile)
- e non richieda troppo tempo di calcolo

Costruire euristiche

Problema rilassato

Considerare un problema con meno restrizioni sulle azioni possibili.



Problema rilassato

Considerare un problema con meno restrizioni sulle azioni possibili.

- Grafo dello spazio degli stati del problema rilassato è un **supergrafo** di quello dello spazio degli stati del problema originale
- Aggiunge archi al grafo dello spazio degli stati
 - Qualsiasi soluzione ottima nel grafo originale è soluzione anche del problema rilassato

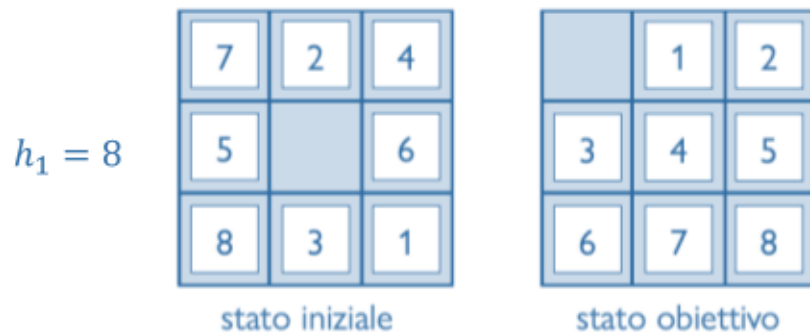
Il costo di una soluzione ottima per un problema rilassato è un'euristica ammissibile per il problema originale.

Problema rilassato: esempio

Eliminando restrizioni dai vincoli del problema

Se le regole fossero diverse...

h_1 = numero di tasselli fuori posto
(spazi vuoti non inclusi).



se ogni tassello potesse muoversi
in qualsiasi posizione invece che
solo in uno spazio vuoto
adiacente.

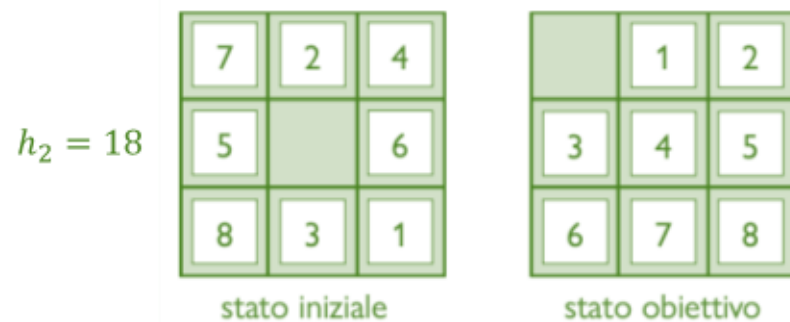
Problema rilassato: esempio

Eliminando restrizioni dai vincoli del problema

Se le regole fossero diverse...

h_2 = somma delle distanze di tutti i tasselli dalla loro posizione corrente alla configurazione obiettivo.

Distanza Manhattan (solo in orizzontale e verticale).



se ogni tassello potesse muoversi ovunque tra le caselle adiacenti, anche se già occupate (non solo nello spazio vuoto).

Massimizzazione

Se abbiamo un insieme di euristiche ammissibili $h_1, \dots, h_m$ e nessuna domina le altre.

Costruiamo $h(n) = \max\{h_1(n), \dots, h_m(n)\}$

- se $h_1, \dots, h_m$ sono ammissibili, anche h lo è
- h domina tutte le altre

Euristiche da sottoproblemi

*	2	4
*		*
*	3	1

stato iniziale

	1	2
3	4	*
*	*	*

stato obiettivo

Es. considerare solo i tasselli 1, 2, 3 e 4, e lo spazio vuoto nella posizione corretta come soluzione; gli altri tasselli possono essere in posizione qualsiasi.

Euristiche da sottoproblemi

*	2	4
*		*
*	3	1

stato iniziale

	1	2
3	4	*
*	*	*

stato obiettivo

Es. considerare solo i tasselli 1, 2, 3 e 4, e lo spazio vuoto nella posizione corretta come soluzione; gli altri tasselli possono essere in posizione qualsiasi.

Il **costo della soluzione ottimale di un sottoproblema** è una sottostima del costo del problema originale

Quindi è un'euristica ammissibile

Database di patterns

Idea: memorizzare i costi esatti delle soluzioni di ogni possibile istanza di un sottoproblema

- es. 1-4 tasselli: $\approx 15k$ patterns
- Possiamo avere un'euristica ammissibile h_{DB} per ogni stato incontrato durante la ricerca, estraendo dal database la configurazione corrisp. del sottoproblema

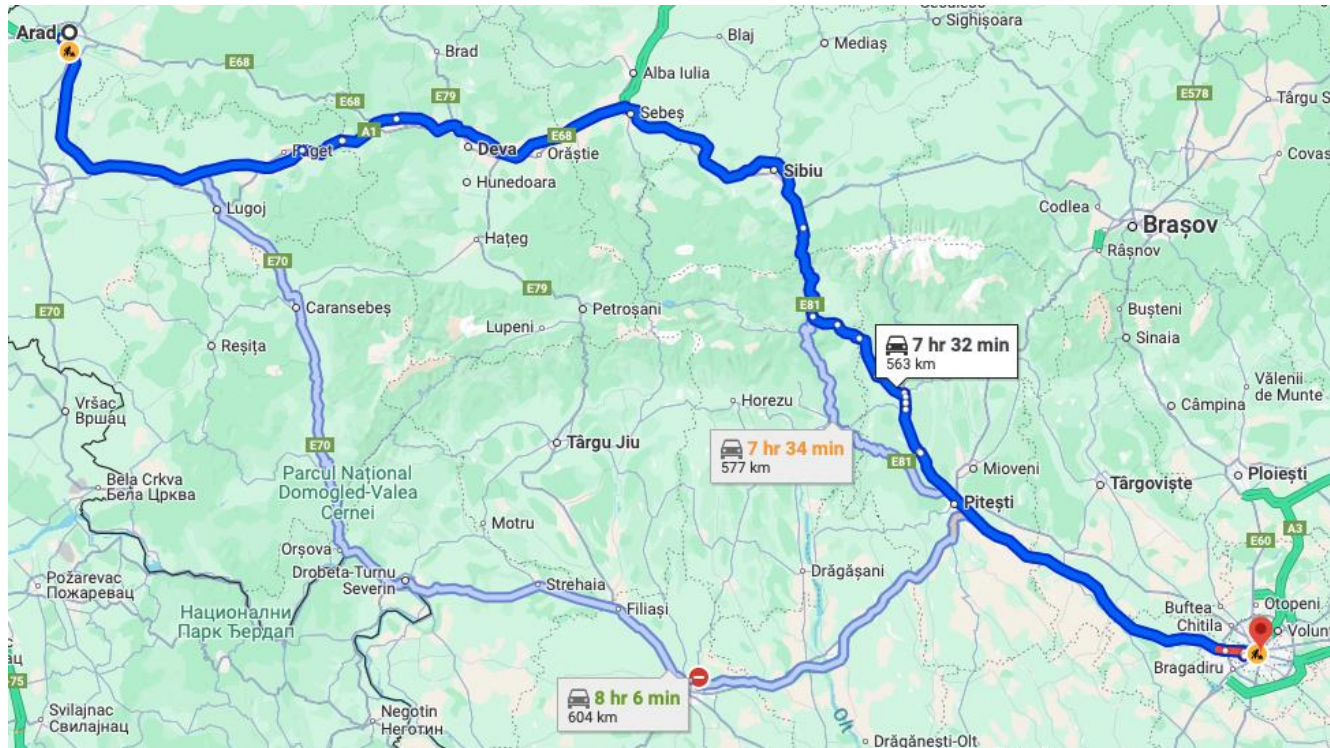
Sottoproblemi multipli

- Possiamo individuare più sottoproblemi di uno stesso problema di partenza, ottenendo diverse euristiche ammissibili
 - es., 1-2-3-4, 5-6-7-8, 2-4-6-8, etc
- Possiamo usare il *max* tra le euristiche individuate dai diversi sottoproblemi
- Non possiamo sommarle
 - le soluzioni ai sottoproblemi possono interferire
 - ✓ se sposto dei tasselli per risolvere 1-2-3-4, sposterò anche dei tasselli che contribuiscono a risolvere 5-6-7-8

Database di pattern disgiunti

- Idea: nel calcolo del costo di una soluzione per un sottoproblema considero solo il costo delle azioni che coinvolgono i tasselli corrispondenti
 - nel calcolo del costo della soluzione per 1-2-3-4, elimino il costo delle mosse che non coinvolgono i tasselli 1-2-3-4
- DB di pattern disgiunti: si possono sommare i costi ottenendo una euristica ammissibile
 - estremamente efficace: gioco del 15 in pochi ms

Mappe reali



- **Pre-calcolo** di alcuni costi di cammino ottimi.
 - anche se richiede tempo può essere svolto una volta sola e in seguito può essere ammortizzato su miliardi di richieste
- **Landmarks**: memorizzare i costi rispetto a **punti di riferimento** rilevanti
 - anche usando conoscenze specifiche come le strade più frequentate

Apprendere dall'esperienza

Idea: risolvere istanze del problema e **imparare dalle soluzioni**.

- Ogni soluzione ottima per un'istanza del problema fornisce una coppia di esempio **<obiettivo, cammino>**
 - oppure $\langle \text{stato}, h^* \rangle$
- Da questi esempi posso usare un **algoritmo di apprendimento** per costruire una funzione h che possa approssimare il costo del cammino reale per altri stati che dovessero presentarsi nella ricerca
 - apprendere una approssimazione della funzione euristica, si rischia l'inammissibilità

Apprendere dall'esperienza

Conveniente fornire caratteristiche (features) dello stato che siano rilevanti per predire il valore dell'euristica

es. possiamo descrivere lo stato in termini di **numero di tasselli fuori posto** e/o **numero di tasselli adiacenti che sono adiacenti anche nella soluzione**

$$h(n) = c_1 x_1(n) + c_2 x_2(n)$$

appresi da un algoritmo di apprendimento



Conclusioni

- **Ricerca informata**: accesso a una funzione euristica $h(n)$ che stima il costo di una soluzione a partire da n .
- **Ricerca best-first greedy** espande i nodi con $h(n)$ minima.
 - non è ottima, ma spesso efficiente
- **Ricerca A*** espande i nodi con $f(n) = g(n) + h(n)$ minima.
 - A* è completa e ottima, purché $h(n)$ sia ammissibile
 - La complessità spaziale è un problema

Conclusioni

- **Ricerca IDA***: versione ad approfondimento iterativo di A*
 - migliora il costo spaziale
- **RBFS** e **SMA*** usano una quantità di memoria limitata
 - dato tempo sufficiente possono risolvere problemi che A* non consentirebbe di risolvere per esaurimento di memoria
- **Beam search**: limite alla dimensione della frontiera
 - incompleta e subottima, ma soluzioni ragionevolmente buone ed eseguita velocemente
- **A* pesata**: focalizza la ricerca verso un obiettivo
 - espande meno nodi, a scapito dell'ottimalità

Conclusioni

Le prestazioni degli algoritmi di ricerca informata dipendono dalla **qualità della funzione euristica**.

Costruire buone euristiche

- ✓ rilassando il problema originale
- ✓ memorizzando in un database di patterns i costi delle soluzioni
- ✓ identificando landmarks
- ✓ apprendendo dall'esperienza